

Bank Account Management System in Java

Aim / Objective

To design and implement a Bank Account Management System in Java using inheritance, demonstrating object-oriented programming concepts such as attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.

Description

The Bank Account Management System uses a base Account class to store common account details and perform common transactions. SavingsAccount and CurrentAccount inherit from Account and add their own features. SavingsAccount calculates interest, while CurrentAccount overrides the withdraw() method to support an overdraft limit.

System Requirements

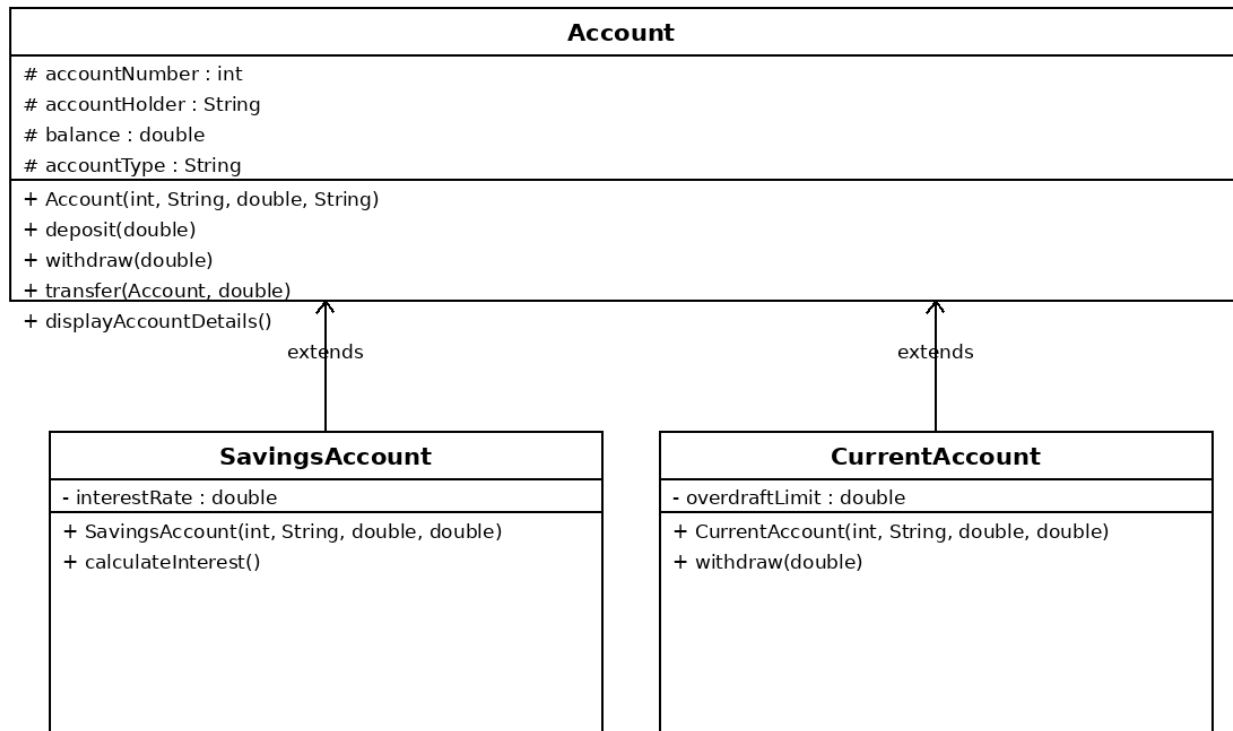
Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse.

Software: Java Development Kit (JDK), Java compiler, Java Runtime Environment (JRE), and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create the Account class with account number, account holder name, balance, and account type.
3. Define the Account constructor and transaction methods.
4. Create SavingsAccount by extending Account and add interestRate.
5. Define calculateInterest() in SavingsAccount.
6. Create CurrentAccount by extending Account and add overdraftLimit.
7. Override withdraw() in CurrentAccount to allow withdrawal within the overdraft limit.
8. Create SavingsAccount and CurrentAccount objects.
9. Perform deposits, withdrawals, interest calculation, and fund transfer.
10. Display account details before and after the transactions.
11. Display the final balances and stop the program.

UML Diagram



Source Code

```

class Account {
    protected int accountNumber;
    protected String accountHolder;
    protected double balance;
    protected String accountType;

    Account(int accountNumber, String accountHolder,
            double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
        this.accountType = accountType;
    }

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }

    void transfer(Account account, double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            account.balance = account.balance + amount;
            System.out.println("Transferred: " + amount);
        } else {
            System.out.println("Insufficient balance for transfer");
        }
    }
}
  
```

```

    }
}

void displayAccountDetails() {
    System.out.println("Account Number : " + accountNumber);
    System.out.println("Account Holder : " + accountHolder);
    System.out.println("Account Type : " + accountType);
    System.out.println("Balance : " + balance);
}
}

class SavingsAccount extends Account {
    private double interestRate;

    SavingsAccount(int accountNumber, String accountHolder,
        double balance, double interestRate) {
        super(accountNumber, accountHolder, balance, "Savings");
        this.interestRate = interestRate;
    }

    double calculateInterest() {
        return balance * interestRate / 100;
    }
}

class CurrentAccount extends Account {
    private double overdraftLimit;

    CurrentAccount(int accountNumber, String accountHolder,
        double balance, double overdraftLimit) {
        super(accountNumber, accountHolder, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Withdrawal limit exceeded");
        }
    }
}

public class BankAccountDemo {
    public static void main(String[] args) {

        SavingsAccount savings =
            new SavingsAccount(101, "Rahul", 10000, 5);

        CurrentAccount current =
            new CurrentAccount(102, "Ravi", 5000, 3000);

        System.out.println("===== BEFORE TRANSACTIONS =====");
        savings.displayAccountDetails();
        System.out.println();
        current.displayAccountDetails();

        System.out.println("\n===== TRANSACTIONS =====");

        savings.deposit(2000);
        savings.withdraw(1000);

        double interest = savings.calculateInterest();
        savings.deposit(interest);
        System.out.println("Interest Added: " + interest);
    }
}

```

```

        current.withdraw(7000);

        savings.transfer(current, 2000);

        System.out.println("\n===== AFTER TRANSACTIONS =====");
        savings.displayAccountDetails();
        System.out.println();
        current.displayAccountDetails();
    }
}

```

Compilation and Execution Commands

Save as: BankAccountDemo.java
 Compile: javac BankAccountDemo.java
 Execute: java BankAccountDemo

Output Screens

Possible Output – Account Details Before Transactions

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\student\Desktop\Java-Projects> javac BankAccountDemo.java
PS C:\Users\student\Desktop\Java-Projects> java BankAccountDemo

===== BEFORE TRANSACTIONS =====
Account Number : 101
Account Holder : Rahul
Account Type   : Savings
Balance       : 10000.0

Account Number : 102
Account Holder : Ravi
Account Type   : Current
Balance       : 5000.0

```

Possible Output – Transactions and Final Balances

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

===== TRANSACTIONS =====
Deposited: 2000.0
Withdrawn: 1000.0
Deposited: 550.0
Interest Added: 550.0
Withdrawn: 7000.0
Transferred: 2000.0

===== AFTER TRANSACTIONS =====
Account Number : 101
Account Holder : Rahul
Account Type   : Savings

```

Result

Thus, the Bank Account Management System was implemented successfully to demonstrate inheritance, constructors, object creation, method overriding, deposits, withdrawals, interest calculation, fund transfers, and account management using Java.